SYSTEM DESIGN SERIES

200+ AI SYSTEM DESIGN INTERVIEW QUESTIONS

- Master **real-world architectures** .
- Clear **senior AI engineer interviews** with confidence.

SYSTEM	RAG & LLM	PROD
SCENARIOS	ARCHITECTURES	SCALE & COST

Swipe to explore scenarios →

You've been asked to add an AI feature to an existing product, but the PM only says "make it smarter." Walk through every clarifying question you would ask before writing a single line of design.

"Make it smarter" is a product wish, not a requirement. Before designing anything, I need to convert that wish into a concrete problem statement with measurable outcomes. Here's exactly how I'd do that.

Questions about the problem:

- What specific user behavior or pain point is driving this request? What are users currently doing manually?
- What does "smarter" mean in this context? Faster responses? More accurate results? Personalized recommendations?
- How are users currently solving this problem? Are they working around a limitation, or is this net-new capability?

Questions about users and usage:

- Who are the primary users? Are they technical or non-technical?
- How often will they interact with this feature? Real-time queries, daily batch, or event-triggered?
- What does a successful interaction look like from the user's perspective? Show an example of a "good" output?

Questions about data and quality:

- What data does the system have access to? Is it clean and labeled?
- How do we measure success? Is there a baseline we're trying to beat?
- What's the error tolerance? If the model is wrong 5% of the time, is that acceptable or catastrophic?

Getting answers to these before writing a single line saves weeks of rework. The biggest design mistakes I've seen all trace back to assumptions made at this stage.

You're given the goal: "improve user experience using AI." How do you break this down into a concrete, scoped system with measurable outcomes?

"Improve user experience using AI" is a direction, not a specification. My job is to decompose it until it becomes buildable and testable.

Step 1: Identify the friction point

I'd start by finding the specific moment in the user journey where the experience breaks down. Is it that users can't find what they're looking for? Are they doing repetitive work? You can't improve user experience in general. You can improve a specific step in a specific workflow.

Step 2: Translate the UX goal into an AI task type

- Users can't find relevant content: this is a **retrieval** problem.
- Users are getting one-size-fits-all responses: this is a **personalization** problem.
- Users are doing repetitive data entry: this is an **extraction or automation** problem.

Step 3: Define measurable outcomes

For each task type, I'd define at least one leading metric (measures system output quality) and one lagging metric (measures actual user impact).

Step 4: Define what "done" looks like

Set a specific target: "reduce average time-to-answer from 3 minutes to under 45 seconds." A target without a number isn't a target.

Step 5: Scope the MVP

Define the narrowest version of this that delivers measurable value. Pick one user type, one workflow, one data source. Get a number. Expand from there.

What You'll Cover Inside

VOLUME 1
Foundations
System thinking, trade-offs

VOLUME 2
LLM Systems
Prompts, context, pipelines

VOLUME 3
RAG Systems
Retrieval, ranking, optimization

VOLUME 4
Agent Systems
Workflows, tools, memory

VOLUME 5
Production Systems
Scaling, latency, cost

VOLUME 6
Real Scenarios
ChatGPT, RAG, voice AI

A traditional backend engineer joins your AI team. What are the three biggest mindset shifts they need to make?

Traditional backend engineers are used to deterministic systems where code is either right or wrong. AI systems operate under a completely different paradigm.

Shift 1: From determinism to probabilistic thinking

Traditional backend systems return identical outputs for identical inputs. In AI systems, output varies. Quality is measured statistically. You evaluate output distributions, not binary pass/fail assertions.

Shift 2: From code correctness to data quality

Backend engineers look for bugs in code. In AI systems, the code is often fine—the problem is bad data, stale retrieval indexes, or misrepresentative evaluation datasets.

Shift 3: From binary pass/fail to graceful degradation

You can't build an AI system that is always right. You have to design for the fact that the model will sometimes be wrong or slow, and build fallbacks so the system fails gracefully.

Your AI system produces slightly different outputs for the same input. How does this non-determinism affect downstream components?

Non-determinism trickles through the entire stack. Here's what breaks and how to handle it.

Components that break without careful handling:

- **Caching layers:** Traditional caches assume same input = same output. Caching LLM responses guarantees variance reduction, which might mean locking in bad responses.
- **Deterministic testing:** Test suites asserting exact string matches will fail constantly. You need semantic assertions or an evaluator model.
- **Downstream integrations:** If a downstream service expects structured output, a slightly different string formatting breaks parsing.
- **Audit and logging:** You can't reproduce a bug simply by re-running the same input.

Practical design responses:

- Use temperature = 0 for paths where determinism matters (classification, extraction).
- Log prompt version, model ID, and seed alongside every response.
- Build schema validation at every stage boundary.

After a model upgrade, accuracy improves by 15% but p99 latency doubles. How do you decide what to prioritize?

This isn't a technical question—it's a product and user experience trade-off.

The real-time chat interface:

Users are waiting. If p99 latency goes from 1.5s to 3s, user drop-off will spike. The 15% accuracy gain won't matter if users abandon the session. I'd block the upgrade for real-time paths until latency is brought down via streaming or speculative decoding.

The batch reporting pipeline:

Reports run overnight. A 2-hour job taking 4 hours has zero user impact. The 15% accuracy gain directly improves business decision quality. I'd roll out the upgrade immediately for batch paths.

The general principle:

Never evaluate latency and accuracy in isolation. Map them against user tolerance per use case, and route traffic to different model tiers accordingly.

You're launching an AI feature in 4 weeks. How do you define success metrics before launch?

Metrics defined after launch are storytelling, not engineering. You need them defined early to build the telemetry to measure them.

Pre-launch metric framework:

For every metric, define: 1. What data to log. 2. Baseline value. 3. Target value at Day 30, 60, 90.

Three metric tiers:

- **System quality:** Output accuracy, hallucination rate, p95/p99 latency, cost per query.
- **User behavior:** Feature adoption, completion rate, retry/edit rate (indicates poor output).
- **Business impact:** Support ticket volume, retention lift, workflow completion time.

How they evolve post-launch:

Week 1: Watch system metrics to prevent fires. Month 1: Optimize user behavior metrics. Month 3: Evaluate business impact against ROI.

A junior engineer defines requirements as: "the model should be accurate and fast." What's wrong with this?

"Accurate and fast" is an ambition, not a spec. You can't design against it, test against it, or know when you're done.

"Accurate" is undefined:

Accurate on what dataset? Measured how (Precision, Recall, F1, exact match)? What error distribution is acceptable? (False positives vs false negatives have very different business costs).

"Fast" is equally vague:

Fast for median user or 99th percentile? Under what load? Fast for time-to-first-token or total response time?

How I'd rewrite it:

"The extraction pipeline must achieve ≥92% F1 score on our golden eval dataset (500 edge-case docs). p95 end-to-end latency must be <800ms at a load of 100 concurrent requests."

Your system must respond in under 150ms end-to-end. Walk through how this constraint shapes design.

A 150ms budget eliminates standard LLM architectures. Here's how every layer is forced to adapt.

Latency budget breakdown:

Network overhead: 30ms. Pre/Post-processing: 20ms. Retrieval: 30ms. Inference budget left: ~70ms.

Model choice:

API-based LLMs (GPT-4, Claude) take 500ms-3s just for TTFT. They're ruled out. Must use small, self-hosted models (1B-7B parameters) or fine-tuned BERT-style models running on dedicated GPUs.

Retrieval constraints:

Vector search over millions of docs takes 50–100ms. Must use in-memory vector indexes (HNSW in RAM) or pre-computed embeddings cached in Redis.

Architectural rule:

No sequential model calls. Everything must be parallelized, cached, or pre-computed.

Under load, one pipeline component will bottleneck first. How do you identify which one before production?

Don't guess under load—profile systematically before shipping.

Step 1: Isolate and profile each component

Benchmark embedding generation, vector DB lookup, reranking, and LLM inference independently. Measure throughput (RPS) and latency at increasing concurrency levels for each.

Step 2: Find the capacity ceiling

Identify which component hits 100% GPU/CPU utilization or queue saturation first as concurrency scales up. In 90% of RAG pipelines, LLM inference or reranking bottlenecks first due to compute limits.

Step 3: Run end-to-end load testing

Simulate production traffic patterns (spiky vs steady). Monitor queue depths between pipeline stages. The stage with a backing-up input queue is your bottleneck.

Step 4: Design mitigations

Add auto-scaling, dynamic batching, or fallback model routing specifically for the bottleneck component.

How does explicitly mapping data flow at each stage change the quality of your design?

Data flow mapping transforms hand-wavy AI ideas into rigorous system engineering.

What explicit data flow mapping forces:

- **Strict input/output schemas:** Defines exact JSON structures, data types, and token limits between pipeline stages.
- **Clear error handling:** Forces you to handle edge cases (empty search results, malformed LLM outputs, context window truncation) at each boundary.
- **Telemetry points:** Identifies exactly where to log inputs, outputs, latencies, and costs for debugging and observability.
- **Decoupled component contracts:** Allows team members to work on retrieval, orchestration, and generation independently without breaking interfaces.

The result:

A system that is testable at every boundary, observable in production, and modular enough to swap out models or DBs with zero downstream breakage.

You've been asked to design a system where core logic is "call GPT-4 and return the result." Why is this not a system design?

An API call is a prototype. System design is everything that makes that call reliable, secure, scalable, and affordable.

What production system design actually requires:

- **Guardrails & Safety:** Prompt injection defense, input validation, output PII scrubbing, toxicity moderation.
- **Context Management:** Multi-turn history pruning, token budget enforcement, dynamic prompt assembly.
- **Resilience:** Fallback model routing when GPT-4 is down/rate-limited, exponential backoff retries, circuit breakers.
- **Performance & Cost:** Semantic caching layer (Redis/GPTCache), request deduplication, async streaming.
- **Observability:** Tracing prompt inputs, token usage, cost tracking, evaluation logging.

Without these 5 layers, you don't have a system—you have a fragile wrapper around someone else's API.

You need to design a system for legal document review. What are the boundaries and contracts?

High-stakes domains require explicit system boundaries and zero-trust verification contracts.

System boundaries:

The AI system NEVER provides legal advice or final decisions. Its boundary is strictly bounded to risk flag extraction, clause comparison, and citation mapping for human attorneys.

Data contracts at key boundaries:

- **Ingestion contract:** Document parser must preserve exact page numbers, line numbers, and bounding box coordinates for source grounding.
- **Extraction contract:** Extracted clauses must include confidence scores and exact verbatim snippet pointers. Zero ungrounded summaries allowed.
- **Evaluation contract:** Hallucination rate must be 0% on factual extraction; any uncertain output defaults to "human review needed" flag.

Human-in-the-loop requirement:

Every AI output requires a 1-click UI path for attorneys to verify source text against original PDF highlights.

Your system has retrieval, orchestration, and generation. Due to a budget cut, one must be simplified. Trade-off strategy?

When cutting budget, evaluate which simplification causes the least degradation to end-user utility.

Option A: Simplify Generation

Swap GPT-4 ($30/M tokens) for GPT-3.5 or Haiku ($0.50/M tokens). User impact: Slightly less natural prose, but factual accuracy remains high if retrieval is solid. Huge cost savings (90%+).

Option B: Simplify Orchestration

Replace multi-step agent loops with single-shot RAG. User impact: Can't handle complex multi-turn workflows, but simple Q&A stays fast and reliable. Eliminates 70% of API call overhead.

Option C: Simplify Retrieval

Replace hybrid vector + keyword search with basic BM25. User impact: Retrieval quality drops sharply, causing hallucinations and missed context.

Recommended strategy:

Simplify generation or orchestration first. NEVER degrade retrieval—bad context makes even the smartest model produce useless output.

Your prototype works flawlessly in a notebook. List assumptions it makes which production will invalidate.

Notebooks assume an ideal world. Production is chaos.

Notebook assumptions that fail in production:

- **Data cleanliness:** Notebook uses pristine sample PDFs. Production gets scanned images, corrupted text, multi-gigabyte files, and missing headers.
- **Concurrency & Scale:** Notebook processes one request sequentially. Production receives 500 concurrent requests with spiky load patterns.
- **API Uptime:** Notebook assumes 100% API availability. Production experiences rate limits (429), timeouts (504), and model degradation.
- **Security:** Notebook assumes friendly users. Production sees prompt injection attacks, data exfiltration attempts, and abuse.
- **Cost:** Notebook costs $2 during dev. Production billing explodes exponentially without token caps and caching.

You have a $500/month infrastructure budget to serve 10,000 users. Architect this cheaply.

Step 1: Calculate unit economics

10,000 users × ~3 queries/day = 900,000 queries/month. $500 / 900k = $0.00055 per query budget. GPT-4 is impossible ($0.03/query).

Step 2: Model & API selection

Use ultra-cheap models: Claude 3 Haiku ($0.25/M tokens) or fine-tuned Llama 3 8B on serverless GPUs (modal/runpod) that scale to zero. Budget allocation: $250/mo for inference.

Step 3: Aggressive Caching (The key lever)

Deploy Redis semantic caching. Common queries hit cache (0 model cost, <20ms latency). Aim for 70%+ cache hit rate. Reduces effective queries from 900k to 270k.

Step 4: Vector DB & Hosting

Use embedded vector DB (Chroma/Qdrant in-memory on server) instead of managed Pinecone ($70+/mo). Host backend on serverless (Vercel/Fly.io) under $50/mo.

200+ QUESTIONS LIKE THIS

Master AI System Design & Engineering.

WHAT WE COVERED:

- → 15 Real-World Architecture Scenarios
- → Concrete Trade-offs & Latency Math
- → Production Engineering Guardrails

Save & Repost for your next interview!
Follow Resnur AI for daily system design breakdowns.